Predavanja

ustni izpit: 40%

predstaviteev: 20% (20 minut po 3 osebe)

Vaje

Projekt 40%

Logic in computer science

- Correctness of software/hardware/systems
  (watch Xavier Leroy: In search of sofware prfection)
- Developing proof asistants

Propositional logic (natural deduction proof system)
Simple type theory : Curry-Howard correspondance
(propositions as types)

Propositional logic

propositional formulas ϕ, ψ, θ

propositional variables p

atomic formula:

$$\phi ::= p \mid \psi \wedge \theta \mid \psi \vee \theta \mid \psi \to \theta \mid \neg\theta \mid \bot$$

TAVTOLOGIJA - validna formula (vedno res)

Natural deduction: proof system. Using formal rules we prove a formula ϕ

We can prove formula $\Leftrightarrow$ it is a tavtology

Primer: Proof of $p \wedge q \to p \vee r$

1. Assume $p \wedge q$ true (assumption)
2. p ($\wedge$elimination$_1$)
3. $p \vee r$ ($\vee$introduction$_1$)
4. $p \wedge q \to p \vee r$ ($\to$ introduction)

Za konstruktivno logiko (Brez LEM

$\wedge$

$$\frac{\phi \quad \psi}{\phi \wedge \psi}\wedge i \qquad \frac{\phi \wedge \psi}{\phi}\wedge e_1 \qquad \frac{\phi \wedge \psi}{\psi}\wedge e_2$$

premises

conclusion

$\vee$

$$\frac{\phi}{\phi \vee \psi}\vee i_1 \qquad \frac{\psi}{\phi \vee \psi}\vee i_2 \qquad \frac{\phi \vee \psi \quad \boxed{\begin{matrix}\phi\\ \vdots\\ \theta\end{matrix}} \quad \boxed{\begin{matrix}\psi\\ \vdots\\ \theta\end{matrix}}}{\theta}\vee e$$

$\rightarrow$

$$\frac{\boxed{\begin{matrix}\phi\\ \vdots\\ \psi\end{matrix}}}{\phi \rightarrow \psi}$$

box discharges the assumption ϕ

$$\frac{\phi \rightarrow \psi \quad \phi}{\psi}\rightarrow e$$

modus ponens

$\neg$

$\neg\phi \equiv \phi \rightarrow \bot$

$$\frac{\boxed{\begin{matrix}\phi\\ \vdots\\ \bot\end{matrix}}}{\neg\phi}\neg i \qquad \frac{\neg\phi \quad \phi}{\bot}\neg e$$

$\bot$

$$\frac{\bot}{\phi}\bot e$$

NO FORMUA IN THE BOX gets used outside the box.

Primer $p \wedge (q \vee r) \rightarrow (p \wedge q) \vee (p \wedge r)$

1. $p \wedge (q \vee r)$ assumption
2. p
3. $q \vee r$
4. q assumption
5. $p \wedge q$
6. $(p \wedge q) \vee (p \wedge r)$
7. r assumption
8. $p \wedge r$
9. $(p \wedge q) \vee (p \wedge r)$ $\vee e$ (3) (4-6) (7-9)
10. $(p \wedge q) \vee (p \wedge r)$ $\rightarrow i$ (1-10)

In classical logic

$$\frac{\boxed{\begin{matrix}\neg\phi\\ \vdots\\ \bot\end{matrix}}}{\phi}raa$$

Reductio ad absurdum

DN: prove LEM with raa

$$\frac{}{\phi \vee \neg\phi} \qquad \frac{\dfrac{\dfrac{\dfrac{}{\neg\phi}}{\bot}}{\phi}}{\phi \vee \neg\phi} \qquad \frac{\dfrac{}{\neg\phi}}{\phi \vee \neg\phi}$$

$$\phi \vee \neg\phi$$

A proof of formula ϕ from assumptions Γ is given by sequence of formulas following the rules such that all open assumptions in the proof are from Γ

notation: $\vdash \phi$

Theorem: $\vdash \phi$ (including raa) $\overset{\text{soundnes}}{\underset{\text{completness}}{\Longleftrightarrow}}$ ϕ is tautology

$\Leftrightarrow$ ϕ is true under any assigment of truth validnes to propositional variables that make all formulas in Γ true

Formulas $\phi ::= p \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \to \phi \mid \bot$

Types $A ::= \alpha \mid A \times B \mid A + B \mid A \to B \mid$

$$t : A$$

term type

$$\frac{s : A \quad t : B}{(s, t) : A \times B} \qquad \frac{t : A \times B}{pr_1(t) : A}\times e_1 \qquad \frac{t : A \times B}{pr_2(t) : B}$$

variable

$$\frac{\begin{array}{c} x : A \\ \vdots \\ t : B \end{array}}{\lambda x : A . t : A \to B} \to i \qquad \frac{t : A \to B \quad u : A}{t u : B} \to e$$

term

$$\frac{t : A}{in_1(t) : A + B} \qquad \frac{t : B}{in_2(t) : A + B}$$

$$\frac{s : A + B \quad \boxed{\begin{array}{c} x : A \\ \vdots \\ t : C \end{array}} \quad \boxed{\begin{array}{c} y : B \\ \vdots \\ u : C \end{array}}}{\text{case } s \, (in_1(x : A) \mapsto t \mid in_2(y : B) \mapsto u) : C}$$

Primer:

A derived term of type $A \times B \to B \times A$

1.	$x : A \times B$	assumption	
2.	$pr_1(x) : A$	$\times e_1$	(1)
3.	$pr_2(x) : B$	$\times e_2$	(1)
4.	$(pr_2(x), pr_1(x)) : B \times A$	$\times i$	(3) (2)
5.	$\lambda x : A \times B . (pr_2(x), pr_1(x)) : A \times B \to B \times A$	$\to i$	(1-4)

tega ne potrebujemo

For every type we have introduction rules, eliminatio rules and computation rules

computation rule for rewrite rule (longer arrow)

$\times$: $pr_1(s,t) \longrightarrow s$

$pr_2(s,t) \longrightarrow t$

β-reduction

substitute u for x in t

$\rightarrow$: $(\lambda x{:}A.t)u \longrightarrow t[x:=u]$

(before doing this first rename bound variable in t that are distinct from variables in u)

$+$: $\text{case}\ (in_1(s))\ (in_1(x) \mapsto t \mid in_2(y) \mapsto u) \longrightarrow t[x:=s]$

0 : $\dfrac{t:0}{\text{empty}(t):A}$

DM: Find canonical term of type

$A \times (B+C) \longrightarrow A\times B + A \times C$

Propositional logic ~ Simple Type theory

Predicate logic ~ Dependent type theory
(first order logic

Propositional logic formulas

$$\phi ::= p \mid \phi \wedge \psi \mid \phi \vee \psi \mid \phi \to \psi \mid \bot$$

arity ≥ 0

Predicate logic extends prop. logic with <u>predicates</u> expressing relations, and <u>constants and function</u> symbols expressing elements

arity ≥ 0

Terms express elements

$$t ::= x \mid c \mid f(t_1, \ldots, t_n)$$

constant

function symbol of arity n

Formulas express propositions

$$\phi ::= P(t_1, \ldots, t_n) \mid \phi \wedge \psi \mid \phi \vee \psi \mid \phi \to \psi \mid \bot \mid \forall x \phi \mid \exists x. \phi$$

Example: a language for expressing property of integers in predicate logic

Constants: $0, 1$

function symbols $+$, $\cdot$ of arity 2

example terms z, $x+y$, $x\cdot(y+(z+1))$

predicates $=$, $\leq$

example formulas $x \leq y$, $\exists y \exists z \exists w \exists v.\ x = yy + z\cdot z + w\cdot w + v\cdot v$

free variable

We could also define $\leq$ as

$$x \leq y \Leftrightarrow \exists x. \exists y. \exists v. \exists u.\ y = x + z\cdot z + w\cdot w + v\cdot v + u\cdot u$$

(vsa naravna števila sa vsote štirih kvadratov)

$x+1=0 \quad (x=-1)$

We will interpreate predicate logic in type theory

Prop ... type of propositions

$$\frac{\phi : Prop \quad \psi : Prop}{\phi \wedge \psi : Prop} \wedge \text{form} \qquad \frac{\phi : Prop \quad \psi : Prop}{\phi \vee \psi : Prop} \vee \text{form.}$$

$$\frac{\phi : Prop \quad \psi : Prop}{\phi \to \psi : Prop} \to \text{formation} \qquad \frac{}{\bot : Prop}$$

(1) $p : Prop$ assumption

(2) $q : Prop$ assumption

} open assumption are called a context

(3) $p \wedge q : Prop$ $\wedge$ formation (1) (2)

(4) $q \wedge p : Prop$ $\wedge$ formation (2) (1)

(5) $p \wedge q \to q \wedge p : Prop$ $\to$ formation (3) (4)

$$p : Prop, q : Prop \vdash p \wedge q \to q \wedge p : Prop$$

For predicate logic we also neen quantifiers

Predicate logic

$$\frac{\boxed{\begin{array}{c} x:A \\ \Phi:Prop \end{array}}}{\forall x:A.\,\Phi:Prop}\ \forall\text{-formation} \qquad \frac{\boxed{\begin{array}{c} x:A \\ \Phi:Prop \end{array}}}{\exists x:A.\,\Phi:Prop}\ \exists\text{-formation}$$

(formation, not introduction

①	$p: A \to Prop$	assumption
②	$x:A$	assumption
③	$p\,x : Prop$	$\to e$
④	$\forall x:A.\,px : Prop$	$\forall$-formation (2) (3)

$$p: A \to Prop \vdash \forall x:A.\,px : Prop$$

(Insted of formation rules, we can include conectives and quantifiers as typed constants

$\to, \vee, \wedge : Prop \to prop \to Prop$

$\exists, \forall : (A \to Prop) \to Prop$)

iz Primera: $\forall(x:A.\,p(x))$

Example proof of

$$p : A \to Prop,\ q : A \to Prop \vdash (\forall x : A.\, px) \to (\forall x : A.\, qx) \to (\forall x : A.\, px \wedge qx)$$

3 judgments forms

$t : A$

$\Phi : Prop$ "Φ is prop."

Φ "Φ is true"

1	$p : A \to Prop$	assump.
2	$q : A \to Prop$	assump.
3	$\forall x : A.\, px$	assump
4	$\forall y : A.\, qy$	assump
5	$z : A$	assum
6	$p\,z$	$\forall E$ (3)
7	$q\,z$	$\forall E$ (4)
8	$p\,z \wedge q\,z$	$\wedge I$ (6) (7)
9	$\forall z : A.\, pz \wedge qz$	$\forall I$ (5)-(8)
10	$\forall y : A.\, qy \to \forall z : A.\, pz \wedge qz$	$\to$ (4)-(9)
11	$\forall x : A.\, px \to (\ldots) \to (\ldots)$	$\to$ (3)-(10)

z se ne sme pojaviti v kontekstu preg

$$\frac{\boxed{\begin{array}{c} z : A \\ \vdots \\ \Phi \end{array}}}{\forall z : A.\, \Phi}\ \forall I$$

$$\frac{\forall x : A.\, \Phi \qquad t : A}{\Phi[x := t]}\ \forall E$$

Simply we replace the grammar for every type with a <u>universe</u> Type and formation rules for types

We had: $A ::= \alpha \mid A \times A \mid A + A \mid A \to A \mid 0$

$$\frac{A : \mathrm{Type} \quad B : \mathrm{Type}}{\begin{array}{c} A + B : \mathrm{Type} \\ A \times B : \mathrm{Type} \\ A \to B : \mathrm{Type} \end{array}}$$

Key idea of dependent type theory is types depend on the context

$\mathbb{N}$ set of natural numbers

List $\mathbb{N}$ set of lists (=finite sequences) of nat. nums

Vector $\mathbb{N}$ n set of lists of n nat. numbers of length n

Dependant product

construction $\prod_{n:\mathbb{N}} \text{Vector}\ \mathbb{N}\ n$ | function f that map $n:\mathbb{N}$ and return a value from the set $\text{Vector}\ \mathbb{N}\ n$

examples

$$n \mapsto \overbrace{0 \ldots 0}^{n}$$

$$n \mapsto \underbrace{1, 2, \ldots, n}_{n}$$

$$\frac{\boxed{\begin{array}{l} x:A \\ \vdots \\ B:\text{Type} \end{array}}}{\Pi x:A.\ B:\text{Type}}\ \Pi\text{-formation}$$

type Nat

$$\frac{}{nat : Type}$$ formation rule

$$\frac{}{0 : nat} \qquad \frac{t : nat}{succ\ t : nat}$$ introduction rules

alternative

$succ : nat \to nat$

Elimination and computation rules latter

$$\frac{t : nat \qquad P : nat \to Prop \qquad _ : P0 \qquad \boxed{\begin{array}{c} x : nat \\ \vdots \\ P(succ\ x) \end{array}}}{Pt}$$ Introduction rule

Instead of rule based formulations we can assume rules:

introduction: $0 : nat \qquad succ : nat \to nat$

elimination:

$$\forall P : nat \to Prop \left(P0 \to (\forall x : nat.\ Px \to P(succ\ x)) \to \forall x : nat.\ Px \right)$$

induction

Iz indukcije v matematiki izrazimo recursion theorem

if v have A set and $a \in A$ and $A \to A$ then there exist unique function $\Gamma : \mathbb{N} \to A$ da $\Gamma(0) = a$ $\Gamma(n+1) = f(\Gamma(n))$

Recursion principle

$$R_{nat} : \Pi_{P : nat \to Type} (P0 \to (\Pi_{x:nat}. Px \to P(succ\, x)) \to \Pi x : nat\ Px)$$

recursor

$$R_{nat}\ P\ b\ f\ : \Pi_{x:nat} Px$$

computation rule:

$$R_{nat}\ P\ b\ f\ 0 \longrightarrow b$$

$$R_{nat}\ P\ b\ f\ (succ\ n) \longrightarrow fn(R_{nat}\ P\ b\ f) : P\,succ(n)$$

Example:

addition $\quad add : Nat \to Nat \to Nat$

$$add := \lambda m{:}nat.\ \lambda n{:}nat.$$
$$R_{nat}\ (\lambda_{:}nat.\ Nat)\ m\ (\lambda_{:}nat.\ \lambda x{:}nat.\ succ\ x)\ n$$

$$add\ \overbrace{(succ\ 0)}^{m}\ \overbrace{(succ\ 0)}^{n} \longrightarrow^*$$
$$R_{nat}\ (_{:}nat.\ nat)\ succ\ 0\ \lambda_{:}nat.\ \lambda x{:}nat.\ succ\ x)\ succ\ n$$
$$\longrightarrow (\lambda_{:}nat.\ \lambda x{:}nat.\ succ\ x)\ 0\ ((R_{nat}\ \lambda_{:}nat.\ nat)$$
$$(succ\ 0)\ (\lambda_{:}nat.\ \lambda x{:}nat.\ succ\ x)\ 0\ ))$$
$$\longrightarrow^* succ\ (R_{nat}(\lambda_{:}nat.\ nat)\ (succ\ 0)$$
$$(\lambda_{:}nat.\ \lambda x{:}nat.\ succ\ x)\ 0)$$
$$\longrightarrow succ\ (succ\ 0) = 2 \qquad \ddot{\smile}$$

type List

$$\frac{A \text{ type}}{\text{list}A : \text{Type}}$$ formation

$$\frac{}{\text{nil} : \text{List}\,A} \qquad \frac{a : A \qquad s : \text{List}\,A}{\text{cons}\ s\ a : \text{list}A}$$

nil : list A

cons : $A \to \text{List}\,A \to \text{List}\,A$. alternative

elimination, computation rules

Vector type

$$\frac{A : Type \qquad n : nat}{vector\ A\ n\ Type}$$

$$\frac{}{nil : vector\ A\ 0} \qquad \frac{a : A \qquad as : vector\ A\ n}{cons^{(n)}\ a\ as : vector\ A\ (succ\ n)}$$

alternative constructs:

nil : vector A 0

cons :

$$\prod_{n:nat} A \to Vector\ A\ n \to Vector\ A\ (succ\ n)$$

Vector is dependent type, because it's depends on the value of n

$$Rvector : \prod_{P : \prod_{n:nat} Vector\ A\ n \to Type} P\ 0\ nil \to \prod_{a:A} \prod_{n:nat} \prod_{as:vector\ A\ n} P\ n\ as \to P\ (sup\ n)\ (cons\ n\ a\ as)) \to \prod_{n:nat} \prod_{as:vector\ A\ n} P\ n\ as$$

Computation rules

$$P : \prod_{n:nat} (vector\ A\ n \to Type)$$

$b : P\ 0\ nil$

$f :$

$$Rec_{vector\ A}\ P\ b\ f : \prod_{n:nat} \prod_{as:vec\ A\ n} P\ n\ as$$

$$Rec_{vec}\ P\ b\ f\ 0\ nil \longrightarrow b$$

$$Rec_{vec}\ P\ b\ f\ (succ\ n)\ (cons\ n$$

??

?

nat, List, vectors so induktivní typi

Dependent product type

$$\frac{A \text{ type} \qquad \boxed{\begin{array}{c} x:A \\ \vdots \\ B \text{ type} \end{array}}}{\Pi_{x:A} B : \text{Type}}$$

intuitively: $\Pi_{x:A} B$ type of functions that maps $x:A$ to type B

introduction

$$\frac{\boxed{\begin{array}{c} x:A \\ \vdots \\ t:B \end{array}}}{\lambda x:A.t : \Pi_{x:A} B}$$

elimination

$$\frac{s : \Pi_{x:A} B \qquad t:A}{s\,t : B[t/x]}$$

Ko imamo odvisni produkt, ne potrebujemo $\rightarrow$

$A \rightarrow B := \prod_{x:A} B$, kjer B ne vsebuje x

computation rule: $(\lambda x:A.t)u \rightarrow t[u/x]$

β-reduction

Dependent sum type

$$\frac{A : \mathrm{Type} \qquad \boxed{\begin{array}{c} x : A \\ \vdots \\ B : \mathrm{Type} \end{array}}}{\sum_{x:A} B : \mathrm{Type}}$$

$$\frac{s : A \qquad t : B[s/x]}{(s,t) : \sum_{x:A} B}$$

$$\frac{t : \sum_{x:A} B}{pr_1(t) : A} \qquad \frac{t : \sum_{x:A} B}{pr_2(t) : B[pr_1(s)/x]}$$

$$\left(A \times B := \sum_{x:A} B \right)$$

Types and their elements

0	no elements
$A \times B$	pairs (a,b), $a:A$ $b:B$
$A+B$	$in_1(a)$; $a:A$ or $in_2(b)$; $b:B$
$A \to B$	functions from A to B
$\prod_{a:A} B$	functions from $a:A \to B[a/x]$
$\sum_{a:A} B$	pairs (a,b) where $a:A$ $b:B[a/x]$

The Brower-Heyting-Kolmogorov interpretation of logic

$\bot$ has no proof

proof of $\Phi \wedge \Psi$ is pair (a,b) where a is proof of Φ and b is proof of Ψ

propositions as subterminal types

if $\Phi : Prop$ then $\Phi : Type$

($Prop$ is a subuniverse of $Type$

$Prop$ is closed under Π-type

$$\left(\frac{A\ type \qquad \boxed{\Phi\ Prop}^{x:A}}{\Pi a:A\ B : Prop} \right)$$

if Φ is $Prop$ then Φ is subterminal

– if $a, b : \Phi$ then $a = b$ (equality next week)

Also: $Prop : Type$ – impredicativity

We can take Π as the only constructor

Scott-Prawitz encoding of intuitionistic logic

$\bot := \Pi_{p:prop}\, p$

"for all propositions p, p is true"

$$\frac{a : \Pi_{p:prop} \qquad \Phi : prop}{a\,\Phi : \Phi} \qquad \text{taej} \qquad \frac{\bot \qquad \Phi\ prop}{\Phi}$$

$\Phi \wedge \psi \qquad \Pi_{p:prop}\, (\Phi \to \psi \to p) \to p$

$\Phi \vee \psi \qquad \Pi_{p:prop}\, (\Phi \to p) \to (\psi \to p) \to p$

$\Phi \to \psi$

$\forall x{:}A.\, \Phi \qquad \Pi_{x:A}\, \Phi$

$\exists x{:}A\, \Phi \qquad \Pi_{p:prop}\, (\Pi_{x:A}\, (\Phi \to p)) \to p$

Basic properties

$A : \mathrm{Type}$

$t : A$ — needs proof that $A : \mathrm{Type}$

$s \equiv t : A$ s, t are judgmentaly equal

Derived judgment forms

$\Phi : \mathrm{Prop}$ ... instance of $t : A$

$t : \Phi$

Φ means $_\ \Phi$ (abstract not $t : \Phi$)

Propositional equality

Formation

$$\frac{A \text{ type} \quad s:A \quad t:A}{s =_A t : \text{prop}}$$

$$\frac{t : A}{\text{refl}_t : t =_A t}$$

Intro

Elim:

$$\mathcal{J}_A : \Pi_{C:(\Pi_{x:A}\Pi_{y:A}((x =_A y) \to \text{Type}))} (\Pi_{x:A} C\, x\, x\, (\text{refl}_{A\,x})) \to \Pi_{x:A}\Pi_{y:A}\Pi_{p:(x =_A y)} C\, x\, y\, p$$

Computation:

$$\mathcal{J}_A\, C\, f\, t\, t\, \text{refl}_{A,t} \to f\, t$$

Equality type

sodbena enakost, ne nevaehne

$$\frac{u : B[s/x] \qquad s \equiv t : A}{u : B[t/s]}\ \text{substitution rule}$$

$$\frac{s : A \qquad t : A \qquad s \xrightarrow{*} u \qquad t \xrightarrow{*} u}{s \equiv t : A}$$

SAT-solving (propositional logic)

variable

$\Phi ::= x \mid \neg\Phi \mid \bot \mid \Phi \vee \Psi \mid \top \mid \Phi \wedge \Psi$

other conectives can be defined

We view formula Φ with at most k propositional variables $x_1, \dots, x_k$ as a function

$$[\![\Phi]\!] : \{0,1\}^k \longrightarrow \{0,1\}$$

semantic brackets

- Φ in variables $x_1, \dots, x_k$ is satisfiable if exists $\vec{b} \in \{0,1\}^k$, such that the formula is true on that assigment of bits
- formula is a tautology or valid if it's true for $\forall \vec{s} \in \{0,1\}^k$. $[\![\Phi]\!](\vec{s}) = 1$
- Φ and Ψ are ekwivalent if for $\forall b \in \{0,1\}^k$. $[\![\Phi]\!](\vec{b}) = [\![\Psi]\!](\vec{b})$

b is called satisfying assigment

Conections with the proof system

- Soundness theorem

if we can prove Φ (from no assumptions) in natural deduction, then Φ is valid.

- completeness theorem

if Φ is valid we can prove Φ in nat. deduction

(we need raa or low of excluded middle)

Φ is valid $\Leftrightarrow$ $\neg\Phi$ is not satisfiable

Definitions give rise to "truth table" algorithms, that check Φ for all $\vec{b} \in \{0,1\}^k$

Proof system prove alternative algorithm for validity: search for a proof

Negation normal form

Φ is in neg. normal form (NNF) if the only negations appear directly in front of propositional variables

$$\Phi ::= x \mid \neg x \mid \bot \mid \Phi \vee \Psi \mid \top \mid \Phi \wedge \Psi$$

Algorithm for conversion to nnf

$$\neg(\Phi \vee \Psi) \longrightarrow (\neg\Phi) \wedge (\neg\Psi)$$
$$\neg(\Phi \wedge \Psi) \longrightarrow \neg\Phi \vee \neg\Psi$$
$$\neg\neg\Phi \longrightarrow \Phi$$
$$\neg\bot \longrightarrow \top$$
$$\neg\top \longrightarrow \bot$$

(nondeterministic rewrite algorithm)

Theorem:

For any formula Φ, the rewrite algorithm terminates with the formula Φ^{nnf} that is equivalent to Φ.

Time complexity is $O(|\Phi|)$ ← size of Φ

also $|\Phi^{nnf}| = O(|\Phi|)$

A literal is a prop. variable x or the negation of x

A clause is a finite disjunction of literals

$L_1 \vee \ldots \vee L_n$ or $\bot$

(A coclause in a finite conjunction of literals

$L_1 \wedge \ldots \wedge L_n$ or $\top$)

A formula is in conjunctive normal form if it is a finite conjunction of clauses

$D_1 \wedge \ldots \wedge D_n$ or $\top$ wher D_i is a clause

A formula is in disjunctive normal form

$C_1 \vee \ldots \vee C_n$

One can give an algorithm that converts any formula Φ to Φ^{cnf} (notes)

or to Φ^{dnf}

The SAT problem

Input: is a prop. formula Φ
Output: yes if Φ is satisiable and
no if Φ is not satisfiable

The useful SAT problem

Input: is a prop. formula Φ
Output: yes if Φ satisfiable + a satisfining assignment,
no if not satisfiable

Famously SAT is NP-complete

There is no known efficient in theory algorithm

The DNF SAT problem

Input: formula in DNF
Output: yes or no

There is an easy algorithm in $O(n^2)$

The CNF sat problem (the useful one)
input: formula in CNF
output: yes or no

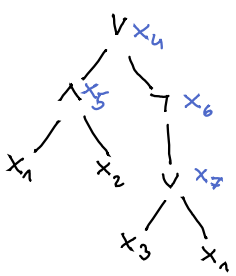
The Tseytin transformation

maps $\Phi(x_1 \dots x_k)$ to $\Phi^T(x_1 \dots x_K)$ where $K \geq k$ such that

- Φ^T is 3-CNF (each contains at most 3 literals)
- $(b_1, \dots b_K) \mapsto (b_1 \dots b_k) : \{0,1\}^K \longrightarrow \{0,1\}^k$ is a bijection satisfying assignments for Φ^T and satisfying assignments
- Φ^T is complete in $O(|\Phi|)$ time

Example

Φ: $(x_1 \wedge x_2) \vee \neg(x_3 \vee x_1)$



$x_4 \Leftrightarrow x_5 \vee x_6$

$x_5 \Leftrightarrow x_1 \wedge x_2$

$x_6 \Leftrightarrow \neg x_7$

$x_7 \Leftrightarrow x_3 \vee x_1$

implikacija $\Rightarrow$ dve implikaciji $\Leftarrow$

1.) $(\neg x_4 \vee x_5 \vee x_6) \wedge (\neg x_5 \vee x_4) \wedge (\neg x_6 \vee x_4)$

2.) $(\neg x_5 \vee x_1) \wedge (\neg x_5 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_5)$

3.) $(\neg x_6 \vee \neg x_7) \wedge (x_7 \vee x_6)$

4) $(\neg x_7 \vee x_3 \vee x_1) \wedge (\neg x_3 \vee x_7) \wedge (\neg x_1 \vee x_7)$

5.) $x_4 =$ 1) $\wedge$ 2) $\wedge$ 3) $\wedge$ 4)

Example: 3-colorability as SAT problem

Let (V, E) be a graph (undirected, irreflexive)

Construct a formula Φ_G, with

prop variables $\{r_v, g_v, b_v \mid v \in V\}$

Φ_G in constituded as CNF formula

as a conjunction of clauses

for every $v \in V$ we include

1.) $r_v \vee g_v \vee b_v$

2) $(\neg r_v \vee \neg g_v) \wedge (\neg g_v \vee \neg b_v) \wedge (\neg r_v \vee \neg b_v)$

(vsak v ima natanko eno barvo

for every (v, v') in E we include

3) $\neg r_v \vee \neg r_{v'}$ $\neg g_v \vee \neg g_{v'}$ $\neg b_v \vee \neg b_{v'}$

Satisfying assignment for Φ_G are in 1:1 correspondence with 3-coloring of G

DPLL (Φ, ρ) ρ assignmet of truth variable

Repeat until is aplicable

- if Φ is [] return ρ as the satisfying assignment
- if Φ containes ()
- if Φ conteines a unit clause L: $\rho' = \rho, L$, DPLL(simp(Φ, L), ρ')

if none of the above aply, we chose som literal L in Φ and continue with $\rho' := \rho, L$

$\Phi' =$ simplify (Φ, L). continue DPLL (Φ', ρ')

if DPLL returns ρ'' then return ρ''

otherwise DPLL $\rho, \neg L$ and $\Phi' =$ simplify$(\Phi, \neg L)$

$[(p \vee q) \wedge (\neg p \vee r) \wedge (\neg g \vee \neg r) \wedge (\neg p \vee g \vee \neg r)]$

choose p

$[r \wedge (\neg g \vee \neg r) \wedge (g \vee \neg r)$ $\qquad p$

unit close r

$[\neg g \wedge g]$ $\qquad p, r$

unit close $\neg g$

$[()]$ $\qquad p, r, \neg g$

consider $\neg p$

$[g \wedge (\neg g \vee \neg r)]$ $\qquad \neg p$

unit clause g

$[\neg r]$ $\qquad \neg p, g$

unti clause $\neg r$

$[]$ $\qquad \neg p, g, \neg r$